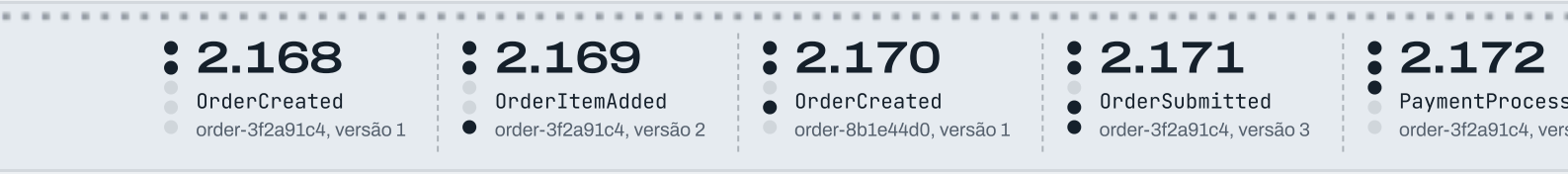


Violet-pixel/
Distributed-Event-Messaging-Engine

Brado

Arquitetura do banco de dados

Event store, outbox, sagas e modelo de leitura: como o banco do Brado está organizado hoje e as regras que a evolução dele deve seguir. Imutabilidade, separação entre escrita e leitura, concorrência, retenção e crescimento.



Versão
1.1.0

Data
Outubro de 2026

Autoria
Copyright (©) 2026 Clara Portes

Sumário

1 **Princípios** 3
Regras que não mudam

2 **Visão geral** 4
Dois bancos, dois propósitos

3 **Convenções** 5
Nomes, tipos, tempo e identificadores

4 **Lado de escrita: eventos** 5
streams, events e snapshots

5 **Lado de escrita: integração** 7
outbox, sagas e inbox

6 **Lado de escrita: broker** 8
Journal e offsets

7 **Lado de leitura** 9
Visões desnormalizadas em EF Core

8 **Transações e concorrência** 10
Como cada escrita é protegida

9 **Imutabilidade e correções** 11
Triggers e eventos compensatórios

10 **Retenção e manutenção** 11
Limpeza, snapshots e backup

11 **Evolução futura** 13
O que o banco deve seguir ao crescer

12 **Checklist e DDL de referência** 14
Implantação e script

Copyright (©) 2026 Clara Portes

1 Princípios

Seis regras sustentam o modelo de dados. Qualquer mudança futura precisa respeitar todas elas.

O log é a verdade

A tabela `events` é a única fonte de verdade do domínio. Visões, snapshots e sagas são derivados e podem ser recalculados.

Append-only

Eventos nunca são alterados nem apagados. Erros se corrigem com novos eventos, e triggers bloqueiam UPDATE e DELETE.

Escrita e leitura separadas

O lado de escrita é normalizado e feito para inserir. O de leitura é desnormalizado para consultar e pode ficar em outro servidor.

Uma transação por decisão

Eventos, versão do stream e mensagens do outbox são gravados juntos. Não existe dual-write entre banco e broker.

Idempotência em toda borda

Ids únicos de evento e de mensagem, inbox por consumidor e checkpoints na transação tornam seguro reprocessar.

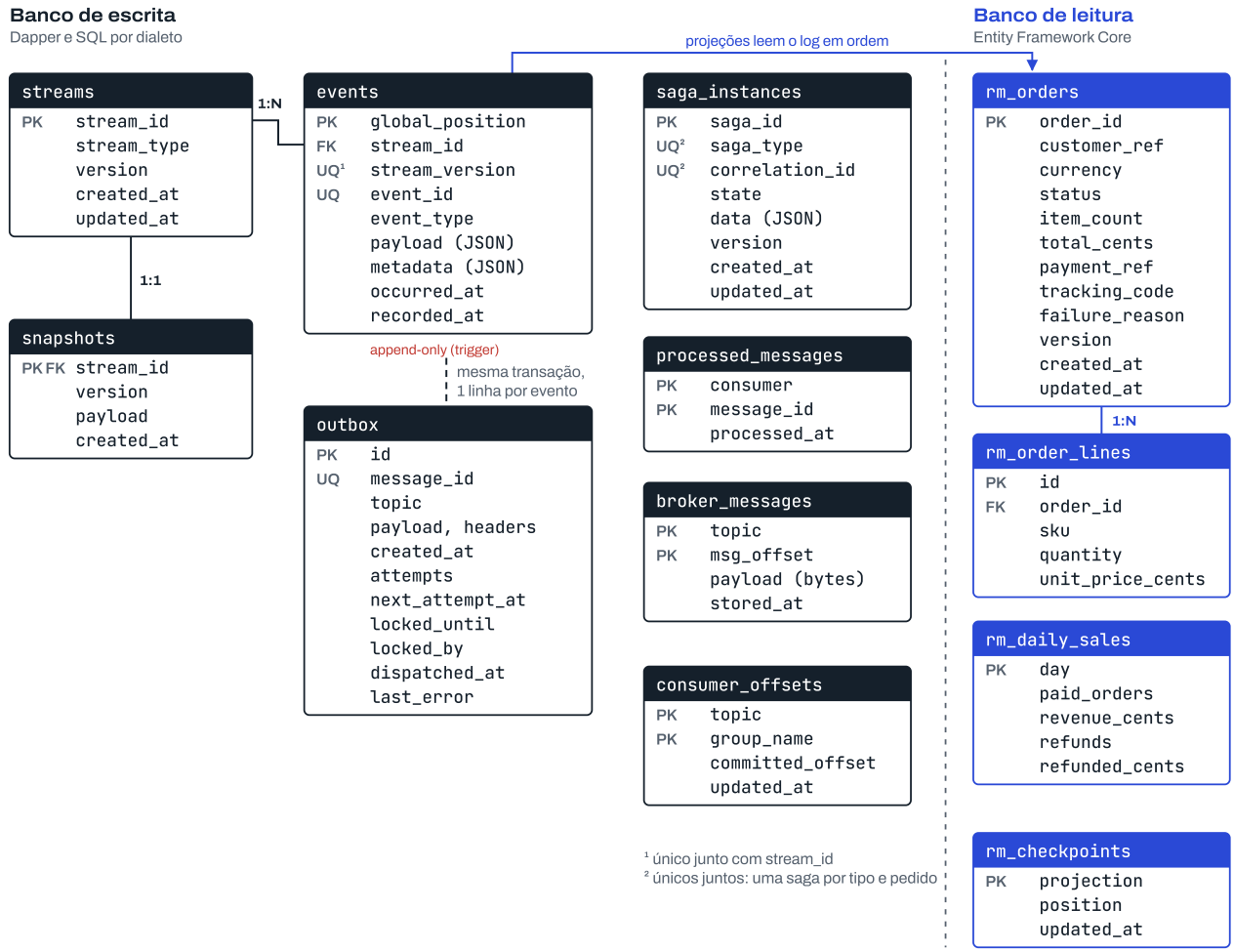
Portabilidade controlada

O mesmo modelo roda em PostgreSQL (produção) e SQLite (desenvolvimento). As diferenças ficam isoladas no dialeto.

Copyright (©) 2026 Clara Portes

2 Visão geral

O Brado usa dois bancos lógicos. Eles podem estar no mesmo servidor, mas nunca compartilham tabelas.



Tabelas atuais. Cabeçalho em tinta: lado de escrita. Cabeçalho em cobalto: lado de leitura, mantido pelas projeções.

Banco	Acesso	Papel	Configuração
Escrita	Dapper e SQL por dialeto	Event store, outbox, sagas, inbox, journal e offsets do broker	`EventStore:Provider` `EventStore:ConnectionString`
Leitura	Entity Framework Core	Visões para consulta e checkpoint das projeções	`ReadModel:Provider` `ReadModel:ConnectionString`

O schema de escrita é criado por um script idempotente (`CREATE ... IF NOT EXISTS`) definido no dialeto. O de leitura é criado pelo EF Core a partir do `ReadModelDbContext`.

Copyright (©) 2026 Clara Portes

3 Convenções

Padrões que valem para todas as tabelas, as de hoje e as que vierem.

Tema	Regra
Nomes	`snake_case` e tabelas no plural. Visões de leitura com prefixo `rm_`; índices `ix_<tabela>_<colunas>`; únicos `uq_`; triggers `trg_`.
Tempo	Sempre UTC. PostgreSQL: `TIMESTAMPTZ`. SQLite: texto ISO-8601 com offset `+00:00`, que ordena corretamente como texto.
Identificadores	UUID v7 gerado pela aplicação (ordenável no tempo), gravado como texto para portabilidade. Chaves sequenciais (`global_position`, `id`) só para ordem física.
JSON	PostgreSQL: `JSONB`, com inserção via `CAST(@p AS jsonb)`. SQLite: `TEXT` com `CHECK (json_valid(...))`. Propriedades em camelCase.
Dinheiro	No modelo de leitura, em centavos (`BIGINT`), para somar e ordenar em qualquer banco. Nos eventos, decimal em JSON.
Inteiros	`BIGINT` para versões, offsets e contadores. Evita estouro e simplifica o mapeamento.
Streams	`<tipo>-<uuid sem hífens>`, por exemplo `order-3f2a91c48b1e44d0...`.

4 Lado de escrita: eventos

Streams, eventos e snapshots formam o núcleo do Event Sourcing.

`streams`

Uma linha por agregado. Guarda a versão atual e é o ponto de trava do controle otimista.

Coluna	PostgreSQL	SQLite	Nulo	Descrição
`stream_id`	`TEXT PK`	`TEXT PK`	não	Identificador do stream.
`stream_type`	`TEXT`	`TEXT`	não	Tipo do agregado (ex.: `order`).
`version`	`BIGINT ≥ 0`	`INTEGER ≥ 0`	não	Quantidade de eventos gravados.
`created_at / updated_at`	`TIMESTAMPTZ`	`TEXT`	não	Criação e último append.

Índice: `ix_streams_type_updated (stream_type, updated_at DESC)`, para listar os streams recentes de cada tipo.

`events`

O log imutável. Cada linha é um fato do domínio, com posição global e versão dentro do stream.

Copyright (©) 2026 Clara Portes

Coluna	PostgreSQL	SQLite	Nulo	Descrição
`global_position`	`BIGINT IDENTITY PK`	`INTEGER AUTOINCREMENT PK`	não	Ordem total, usada pelas projeções e pelo replay.
`stream_id`	`TEXT FK → streams`	`TEXT FK`	não	Stream dono do evento.
`stream_version`	`BIGINT > 0`	`INTEGER > 0`	não	Versão no stream; única junto com `stream_id`.
`event_id`	`TEXT UNIQUE`	`TEXT UNIQUE`	não	UUID v7. Também é o `messageId` no outbox.
`event_type`	`TEXT`	`TEXT`	não	Nome estável do evento (ex.: `PaymentProcessed`).
`payload`	`JSONB`	`TEXT (json_valid)`	não	Dados do evento.
`metadata`	`JSONB`	`TEXT (json_valid)`	não	`aggregateType`, `schemaVersion`, `correlationId`.
`occurred_at / recorded_at`	`TIMESTAMPTZ`	`TEXT`	não	Quando aconteceu e quando foi gravado.

Restrições: `uq_events_stream_version (stream_id, stream_version)` e `uq_events_event_id`. Índice: `ix_events_type`. Triggers impedem UPDATE e DELETE.

snapshots

Coluna	PostgreSQL	SQLite	Nulo	Descrição
`stream_id`	`TEXT PK FK`	`TEXT PK FK`	não	Um snapshot por stream, sempre o mais recente.
`version`	`BIGINT`	`INTEGER`	não	Versão do stream que o snapshot representa.
`payload`	`JSONB`	`TEXT`	não	Estado serializado do agregado.
`created_at`	`TIMESTAMPTZ`	`TEXT`	não	Momento da captura.

O snapshot é gerado a cada `SnapshotEvery` eventos. Na carga, o repositório restaura o snapshot e aplica só os eventos com versão maior. Snapshots são descartáveis: apagar um deles só deixa a carga mais lenta.

Copyright (©) 2026 Clara Portes

5 Lado de escrita: integração

Tabelas que ligam o event store ao broker e coordenam processos longos.

outbox

Coluna	PostgreSQL	SQLite	Nulo	Descrição
`id`	`BIGINT IDENTITY PK`	`INTEGER AUTOINCREMENT PK`	não	Ordem de entrega.
`message_id`	`TEXT UNIQUE`	`TEXT UNIQUE`	não	Igual ao `event_id`; chave de deduplicação nos consumidores.
`topic`	`TEXT`	`TEXT`	não	Tópico de destino (ex.: `orders.events`).
`payload / headers`	`JSONB`	`TEXT`	não	Envelope do evento e cabeçalhos.
`created_at`	`TIMESTAMPTZ`	`TEXT`	não	Gravação, na mesma transação do evento.
`attempts`	`BIGINT`	`INTEGER`	não	Tentativas que falharam.
`next_attempt_at`	`TIMESTAMPTZ`	`TEXT`	não	Próxima tentativa (backoff).
`locked_until / locked_by`	`TIMESTAMPTZ / TEXT`	`TEXT`	sim	Lease do worker que reservou a linha.
`dispatched_at`	`TIMESTAMPTZ`	`TEXT`	sim	Entrega confirmada.
`last_error`	`TEXT`	`TEXT`	sim	Última falha, com até 2.000 caracteres.

Índices parciais: `ix_outbox_pending (next_attempt_at, id) WHERE dispatched_at IS NULL` e `ix_outbox_dispatched (dispatched_at) WHERE dispatched_at IS NOT NULL`. Assim a fila ativa continua pequena mesmo com milhões de linhas entregues.

saga_instances

Coluna	PostgreSQL	SQLite	Nulo	Descrição
`saga_id`	`TEXT PK`	`TEXT PK`	não	Identificador da instância.
`saga_type / correlation_id`	`TEXT`	`TEXT`	não	Únicos juntos: uma saga por tipo e por pedido.
`state`	`TEXT`	`TEXT`	não	Estado atual (`AwaitingPayment`, `Compensating` e outros).
`data`	`JSONB`	`TEXT`	não	Dados do passo (ex.: referência do pagamento).
`version`	`BIGINT`	`INTEGER`	não	Incrementada a cada transição.
`created_at / updated_at`	`TIMESTAMPTZ`	`TEXT`	não	Ciclo de vida; índice em `updated_at DESC`.

Copyright (©) 2026 Clara Portes

processed_messages

Coluna	PostgreSQL	SQLite	Nulo	Descrição
consumer + message_id	TEXT PK composta	TEXT PK composta	não	Mensagem já tratada por um consumidor (inbox).
processed_at	TIMESTAMPTZ	TEXT	não	Momento do processamento.

6 Lado de escrita: broker

Persistência opcional do broker, para sobreviver a reinícios (`Broker:PersistMessages`).

broker_messages

Coluna	PostgreSQL	SQLite	Nulo	Descrição
topic + msg_offset	TEXT + BIGINT PK	TEXT + INTEGER PK	não	Mensagem por tópico e offset.
payload	BYTEA	BLOB	não	Bytes publicados, sem interpretação.
stored_at	TIMESTAMPTZ	TEXT	não	Momento do append no broker.

Gravada em lotes de até 512 mensagens por transação, a partir de um `Channel` (group commit), com `ON CONFLICT DO NOTHING`. A cada 500 lotes, as mensagens fora da janela `RetainedMessagesPerTopic` são removidas.

consumer_offsets

Coluna	PostgreSQL	SQLite	Nulo	Descrição
topic + group_name	TEXT PK composta	TEXT PK composta	não	Grupo de consumo.
committed_offset	BIGINT	INTEGER	não	Menor offset ainda não confirmado.
updated_at	TIMESTAMPTZ	TEXT	não	Última gravação: a cada 5 s e no desligamento.

> **Recuperação.** Na inicialização o broker relê o final de cada tópico e os offsets, e os grupos voltam a entregar a partir do último offset confirmado. Mensagens confirmadas depois da última gravação podem ser reentregues; por contrato, os consumidores são idempotentes.

Copyright (©) 2026 Clara Portes

7 Lado de leitura

Visões desnormalizadas mantidas pelas projeções e consultadas com `AsNoTracking`.

Tabela	Chave	Colunas principais	Índices
`rm_orders`	`order_id`	customer_ref, currency, status, item_count, total_cents, payment_ref, tracking_code, failure_reason, version, created_at, updated_at	`(status, updated_at)`, `updated_at`
`rm_order_lines`	`id`	order_id, sku, quantity, unit_price_cents	`order_id`
`rm_daily_sales`	`day (DateOnly)`	paid_orders, revenue_cents, refunds, refunded_cents	nenhum
`rm_checkpoints`	`projection`	position, updated_at	nenhum

Regras do modelo de leitura

- Uma consulta, uma tabela. Cada tela deve ser atendida sem joins custosos; se precisar de um, crie uma visão e uma projeção novas.
- O checkpoint anda junto com os dados: as visões e `rm_checkpoints.position` são salvos na mesma transação.
- Idempotência por versão: a projeção ignora eventos com `stream_version` menor ou igual ao `version` da linha.
- Tudo é descartável. Se o formato mudou, apague e reconstrua por replay (`POST /api/projections/rebuild`), sem migração de dados.
- Só as projeções escrevem nas tabelas `rm_`.

Copyright (©) 2026 Clara Portes

8 Transações e concorrência

Cada operação de escrita tem um mecanismo explícito contra condições de corrida.

Append de eventos

```
BEGIN;
SELECT pg_advisory_xact_lock(74201);
SELECT version FROM streams WHERE stream_id = @id FOR UPDATE;
INSERT INTO streams (...) VALUES (...)
  ON CONFLICT (stream_id) DO UPDATE SET version = excluded.version, updated_at =
excluded.updated_at;
INSERT INTO events (...) VALUES (...);
INSERT INTO outbox (...) VALUES (...);
COMMIT;
```

- No SQLite a transação abre com `BEGIN IMMEDIATE`, sem advisory lock e sem `FOR UPDATE`. Se a versão lida for diferente da esperada, a transação é abortada antes de qualquer escrita. `events` e `outbox` recebem uma linha por evento.
- Controle otimista: o comando informa a versão que leu, e qualquer divergência aborta. O dispatcher de comandos repete até 3 vezes, recarregando o agregado.
- Última barreira: `uq_events_stream_version` rejeita duas gravações da mesma versão, mesmo que a checagem anterior seja contornada.
- Ordem global: a advisory lock serializa os appends no PostgreSQL para que `global_position` siga a ordem de commit. Sem ela, uma projeção poderia avançar o checkpoint além de uma transação ainda aberta e pular eventos.

Reserva do outbox

```
UPDATE outbox SET locked_until = @agora + @lease, locked_by = @worker
WHERE id IN (SELECT id FROM outbox
             WHERE dispatched_at IS NULL AND next_attempt_at ⩽ @agora
               AND (locked_until IS NULL OR locked_until < @agora)
             ORDER BY id LIMIT @lote FOR UPDATE SKIP LOCKED)
RETURNING id, message_id, topic, payload, headers, attempts, created_at;
```

Projeções e sagas

As projeções rodam com um semáforo por processo e checkpoint na transação. As sagas usam `uq_saga_correlation` e upsert com incremento de versão, e a inbox impede que a mesma mensagem seja processada duas vezes.

Copyright (©) 2026 Clara Portes

9 Imutabilidade e correções

O histórico é auditável porque não pode ser reescrito.

```
CREATE OR REPLACE FUNCTION brado_events_are_immutable() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'events é append-only: UPDATE/DELETE não são permitidos';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_events_immutable BEFORE UPDATE OR DELETE ON events
    FOR EACH ROW EXECUTE FUNCTION brado_events_are_immutable();
```

No SQLite, dois triggers (`trg_events_no_update` e `trg_events_no_delete`) fazem `RAISE(ABORT, ...)`.

Como corrigir dados

Situação	Abordagem correta
Valor errado registrado	Um novo evento de correção (ex.: `OrderPriceCorrected`), aplicado pelo agregado e pelas projeções.
Operação que não deveria ter acontecido	Um evento compensatório, como `PaymentRefunded` seguido de `OrderCancelled`.
Visão de leitura inconsistente	Rebuild das projeções. Nunca edite as tabelas `rm_` à mão.
Evento com formato antigo	Upcaster na leitura (ver capítulo 11), sem reescrever a linha.

> **Acesso administrativo.** Em produção, o usuário da aplicação deve ter apenas INSERT e SELECT em `events`. O trigger é a segunda linha de defesa.

10 Retenção e manutenção

O que cresce, o que é limpo automaticamente e o que precisa de rotina operacional.

Objeto	Crescimento	Manutenção automática	Rotina recomendada
`events`	Ilimitado (é o histórico)	Nenhuma	Particionar e arquivar faixas antigas (capítulo 11).
`snapshots`	1 linha por stream	Sobrescrito a cada N eventos	Nenhuma.

Copyright (©) 2026 Clara Portes

Objeto	Crescimento	Manutenção automática	Rotina recomendada
`outbox`	1 linha por evento	Entregues há mais de 7 dias são removidos a cada hora	Alertar se os pendentes crescerem.
`processed_messages`	1 linha por mensagem e consumidor	Nenhuma	Remover linhas com mais de 30 dias (janela de reentrega).
`broker_messages`	Até a retenção do tópico	Limpeza a cada 500 lotes	Ajustar `RetainedMessagesPerTopic`.
`rm_*`	Proporcional aos agregados	Projeções	Rebuild quando o formato mudar.

PostgreSQL

- Autovacuum mais agressivo em `outbox` e `processed_messages`, que têm muitas atualizações e remoções: `autovacuum_vacuum_scale_factor = 0.02`.
- `events` só recebe inserções, então o custo de vacuum é baixo. Mantenha `fillfactor` 100.
- Backup contínuo (WAL archiving ou PITR) do banco de escrita. O de leitura dispensa backup.

SQLite

- WAL ligado, `synchronous = NORMAL`, `busy_timeout = 10000` e transações `BEGIN IMMEDIATE`.
- Serve para desenvolvimento, demonstrações e instâncias únicas. Para vários escritores, use PostgreSQL.

Copyright (©) 2026 Clara Portes

11 Evolução futura

Diretrizes para o banco seguir conforme o volume e os requisitos crescerem.

Particionamento de events

Ao passar de dezenas de milhões de linhas, converta `events` em tabela particionada por faixa de `global_position` (ou por mês de `recorded_at`). As restrições únicas passam a incluir a chave de partição, e a unicidade global de `event_id` migra para uma tabela de índice `event_ids(event_id PK, global_position)`.

```
CREATE TABLE events (...) PARTITION BY RANGE (global_position);
CREATE TABLE events_p000 PARTITION OF events FOR VALUES FROM (1) TO (50000001);
CREATE INDEX ix_events_recorded_brin ON events USING BRIN (recorded_at);
```

Versionamento do schema dos eventos

- Todo evento já grava `schemaVersion` em `metadata`. Uma mudança incompatível cria a versão seguinte.
- A leitura usa upcasters encadeados (v1 para v2, v2 para v3) no `EventTypeRegistry`. O banco nunca é reescrito.
- Para renomear um evento, registre um alias do nome antigo. Nunca altere o `event_type` gravado.

Migrações versionadas

- Escrita: scripts numerados (`V001__base.sql`, `V002__particionamento.sql`) com tabela `schema_history`, executados antes dos workers.
- Leitura: migrações do EF Core no lugar da criação automática quando houver mais de uma instância.

Escala de leitura e notificação

- Réplicas de leitura para as visões `rm_` e para replays pesados.
- `LISTEN/NOTIFY` no PostgreSQL para acordar projeções e outbox entre servidores, com o polling como reserva.
- Projeções novas em bancos especializados (busca, séries temporais), alimentadas pelo mesmo log.

Arquivamento

Faixas antigas de `events` vão para armazenamento frio (Parquet), com hash encadeado por lote. Snapshots recentes garantem que nenhum agregado ativo dependa do arquivo.

Dados pessoais (LGPD)

- Eventos não devem levar dados pessoais em claro. Use referências (como `customerRef`) ou campos cifrados.

Copyright (©) 2026 Clara Portes

- Crypto-shredding: uma chave por titular numa tabela separada (`subject_keys`). Apagar a chave torna os dados ilegíveis sem violar a imutabilidade do log.

Multi-tenant e segurança

- Coluna `tenant_id` em todas as tabelas de escrita, compondo chaves e índices, com Row Level Security.
- Papéis distintos: aplicação (INSERT e SELECT em `events`), workers (UPDATE no `outbox`) e leitura (somente `rm_`). Tipos nativos `uuid` quando o SQLite deixar de ser alvo.

12 Checklist e DDL de referência

O que conferir antes de colocar um ambiente no ar, e o núcleo do script do PostgreSQL.

- [ ] Event store e modelo de leitura em bancos separados, ou ao menos em schemas distintos.
- [ ] Usuário da aplicação sem permissão de UPDATE e DELETE em `events` .
- [ ] Backup contínuo (PITR) do banco de escrita, testado com uma restauração.
- [ ] Alertas para outbox pendente, lag dos grupos e falhas de projeção.
- [ ] Rotina de limpeza de `processed_messages` .
- [ ] Fuso do servidor indiferente: a aplicação grava sempre em UTC.
- [ ] Plano de particionamento definido antes de 50 milhões de eventos.

Copyright (©) 2026 Clara Portes

```
CREATE TABLE IF NOT EXISTS streams (
    stream_id TEXT PRIMARY KEY, stream_type TEXT NOT NULL,
    version BIGINT NOT NULL CHECK (version ⩾ 0),
    created_at TIMESTAMPTZ NOT NULL, updated_at TIMESTAMPTZ NOT NULL);

CREATE TABLE IF NOT EXISTS events (
    global_position BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    stream_id TEXT NOT NULL REFERENCES streams (stream_id),
    stream_version BIGINT NOT NULL CHECK (stream_version > 0),
    event_id TEXT NOT NULL, event_type TEXT NOT NULL,
    payload JSONB NOT NULL, metadata JSONB NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL, recorded_at TIMESTAMPTZ NOT NULL,
    CONSTRAINT uq_events_stream_version UNIQUE (stream_id, stream_version),
    CONSTRAINT uq_events_event_id UNIQUE (event_id));

CREATE TABLE IF NOT EXISTS outbox (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    message_id TEXT NOT NULL UNIQUE, topic TEXT NOT NULL,
    payload JSONB NOT NULL, headers JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL, attempts BIGINT NOT NULL DEFAULT 0,
    next_attempt_at TIMESTAMPTZ NOT NULL, locked_until TIMESTAMPTZ NULL,
    locked_by TEXT NULL, dispatched_at TIMESTAMPTZ NULL, last_error TEXT NULL);
CREATE INDEX IF NOT EXISTS ix_outbox_pending ON outbox (next_attempt_at, id) WHERE
dispatched_at IS NULL;
```

O script completo, com snapshots, sagas, inbox, journal, offsets e triggers, está em `src/Brado.EventStore/Dialects/PostgresDialect.cs` e `SqliteDialect.cs`.

Copyright (©) 2026 Clara Portes